

Universidade de Brasília — UnB

Faculdade do Gama — FGA

Orientação a Objetos

Prof. André Luiz Peron Martins Lanna

Trabalho Prático de Orientação a Objetos

Sistema de Vendas e Fidelidade — Cafeteria Geek "Byte & Brew"

Relatório Técnico

Vinicius Evangelista Borges Pinto

Brasília, 05 de julho de 2026

1. Introdução

Este relatório documenta o projeto e a implementação em Java do sistema de vendas e fidelidade da cafeteria temática *Byte & Brew*, desenvolvido para o Trabalho Prático da disciplina de Orientação a Objetos. O sistema gerencia três pilares — produtos, clientes e pedidos — e implementa o programa de fidelidade baseado em XP descrito no enunciado.

As seções a seguir apresentam o diagrama de classes UML (Seção 2), a explicação de cada associação, herança e forma de polimorfismo exigida pelo checklist de conceitos obrigatórios (Seção 3) e a justificativa para as duas exceções customizadas do domínio (Seção 4). O código-fonte está organizado nos pacotes `br.edu.cafeteria.modelo`, `br.edu.cafeteria.servico`, `br.edu.cafeteria.excecao` e `br.edu.cafeteria.app`, conforme exigido no requisito de modularidade do enunciado.

2. Diagrama de Classes UML

2.1. Modelo de Domínio

O núcleo do sistema gira em torno da classe `Pedido`, que associa um `Cliente` (opcional), um `Atendente` (obrigatório) e uma coleção de `ItemPedido`, cada um referenciando um `Produto` concreto (`Comida` ou `Bebida`). O desconto promocional é aplicado por meio da interface `Promocional`, e duas exceções customizadas (`EstoqueInsuficienteException` e `PontosInsuficientesException`) modelam as regras de negócio que podem falhar.

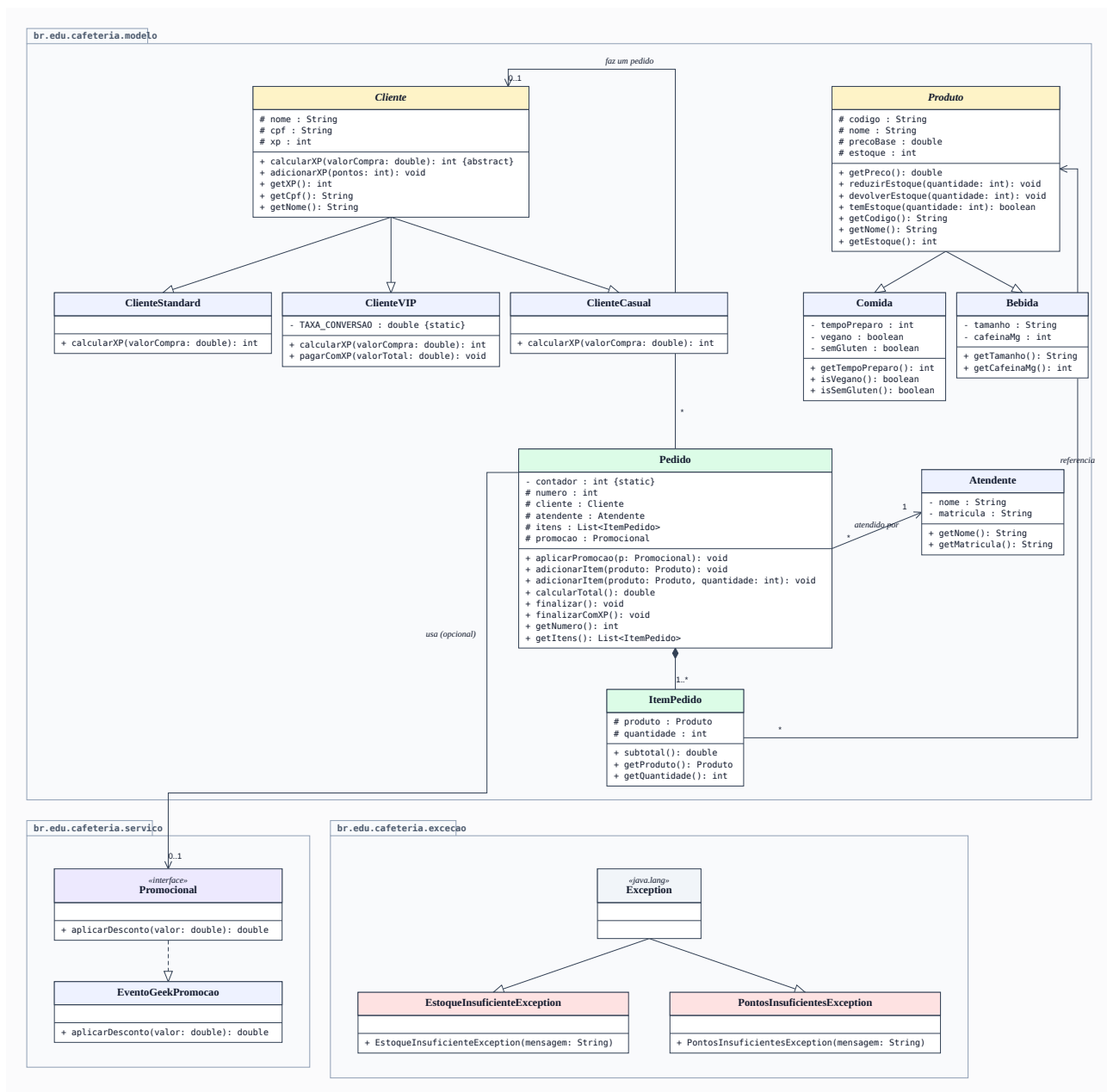


Figura 1 — Diagrama de classes do modelo de domínio.

2.2. Camada de Serviço (Cadastros)

Os três cadastros (`CadastroProduto` , `CadastroCliente` e `CadastroPedido`) seguem o mesmo padrão: mantêm uma coleção da respectiva entidade e oferecem as operações de CRUD exigidas pelo enunciado (cadastrar, pesquisar, atualizar, remover e listar). Essas classes agregam as entidades do modelo de domínio detalhadas na Figura 1.

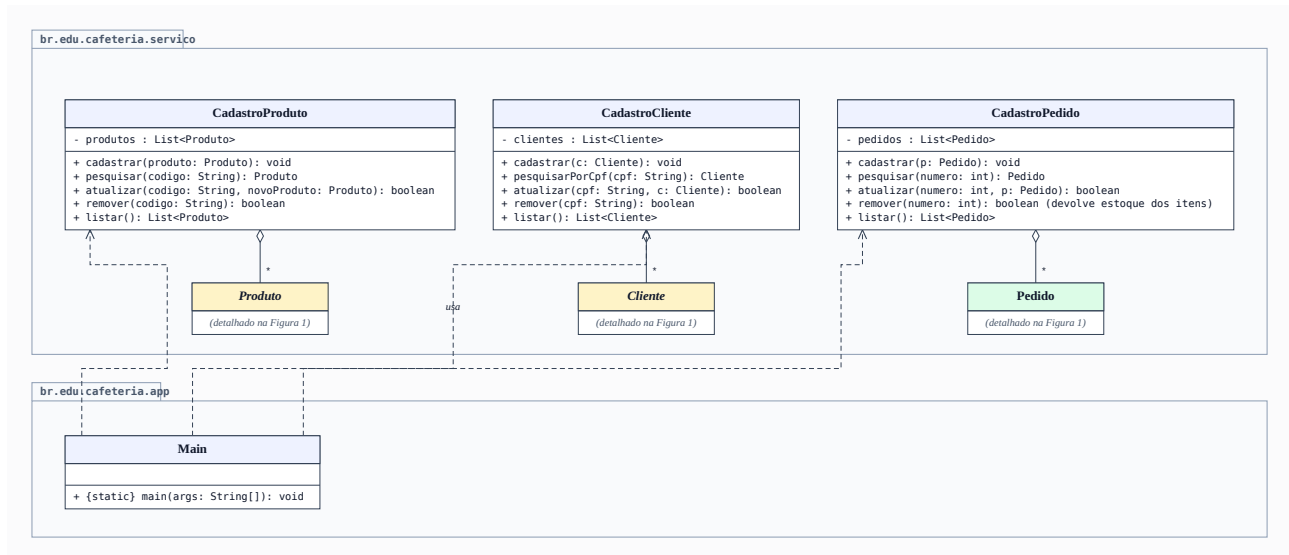


Figura 2 — Diagrama de classes da camada de serviço.

3. Associações, Heranças e Polimorfismos Aplicados

Esta seção percorre, um a um, os itens do checklist de conceitos obrigatórios do enunciado.

3.1. Classes, Objetos e Associações Item 1

`Pedido` é o centro das associações do sistema:

- **Pedido** → **Cliente** (associação, multiplicidade `0..1`): um pedido pode ou não ter um cliente cadastrado (venda avulsa), conforme RF 2.3.
- **Pedido** → **Atendente** (associação, multiplicidade `1`): todo pedido é obrigatoriamente aberto por um atendente.
- **Pedido** ◆— **ItemPedido** (composição, `1` para `*`): cada `ItemPedido` só existe enquanto pertence a um `Pedido` — não faz sentido um item de pedido avulso, fora de qualquer pedido.
- **ItemPedido** → **Produto** (associação, `*` para `1`): cada item referencia o produto comprado e a quantidade solicitada.
- **Pedido** → **Promocional** (associação opcional, via interface): o pedido conhece apenas o contrato `Promocional`, nunca a classe concreta que o implementa.

Vale registrar uma observação de projeto: o enunciado descreve `Pedido` como quem "deve armazenar uma lista de produtos". A implementação guarda, em vez disso, uma `List<ItemPedido>`, em que cada `ItemPedido` encapsula um `Produto` mais a `quantidade` solicitada. Essa escolha foi necessária porque a Seção 2.3 do enunciado exige registrar "o produto e a quantidade desejada" por item — uma lista simples de `Produto` não teria como representar "duas Poções de Mana e um Portal Cake" sem repetir elementos. O polimorfismo de inclusão (ver 3.4) continua presente dentro de `ItemPedido`, que trata qualquer `Produto` de forma genérica.

3.2. Modificadores de Escopo — Estático vs. Dinâmico Item 2

Dois atributos estáticos (de classe, compartilhados por todas as instâncias) sustentam regras de negócio centrais:

```
// Pedido.java
protected static int contador = 1;    // por classe (estatico)
protected int numero;                 // por instancia (dinamico)
```

A cada novo `Pedido` construído, `numero = contador++` captura o valor atual do contador estático e em seguida o incrementa — é assim que a numeração sequencial de pedidos é gerada sem que nenhum objeto precise conhecer os demais.

```
// ClienteVIP.java
protected static final double TAXA_CONVERSAO = 10.0;
```

`TAXA_CONVERSAO` é a mesma para qualquer `ClienteVIP` — não faz sentido que cada cliente VIP tivesse sua própria taxa de conversão, por isso o atributo é estático (e `final`, por ser uma constante do domínio).

3.3. Herança Simples Item 3

Superclasse (abstrata)	Subclasses	O que cada subclasse especializa
Produto	Comida	tempo de preparo, indicadores vegano / sem glúten
	Bebida	tamanho (P/M/G), dosagem de cafeína
Cliente	ClienteStandard	ganha 1 XP por R\$ 1,00 gasto
	ClienteVIP	ganha 2 XP por R\$ 1,00 gasto; pode pagar com XP acumulado
	ClienteCasual	representa a venda avulsa (sem cadastro); nunca acumula XP

`ClienteCasual` é uma adição em relação ao mínimo exigido pelo checklist (que cita apenas `ClienteStandard` e `ClienteVIP`), mas corresponde diretamente ao "Cliente Casual" descrito no enunciado ("Qualquer pessoa pode comprar na cafeteria de forma anônima, como um cliente casual"). Ver a justificativa completa em 3.5.

3.4. Polimorfismo por Inclusão Item 4

`ItemPedido.subtotal()` processa qualquer `Produto` de forma genérica, sem jamais verificar se é uma `Comida` ou uma `Bebida` :

```
// ItemPedido.java
public double subtotal(){
    return produto.getPreco() * quantidade;
}
```

A variável `produto` é declarada com o tipo da superclasse (`Produto`), mas em tempo de execução pode referenciar um objeto `Comida` ou `Bebida` — o método `getPreco()` chamado é resolvido dinamicamente para a implementação herdada de `Produto` (ou eventualmente sobrescrita), sem que `ItemPedido` precise saber qual subtipo concreto está tratando. O mesmo princípio aparece em `Pedido.calcularTotal()` , que soma os subtotais de todos os itens independentemente do tipo de produto envolvido.

3.5. Polimorfismo por Sobrescrita Item 5

`Cliente` declara o método abstrato `calcularXP(double valorCompra)`, sobrescrito de forma diferente em cada subclasse:

```
// Cliente.java
public abstract int calcularXP(double valorCompra);

// ClienteStandard.java (1 XP por R$ 1,00)
public int calcularXP(double valorCompra){
    return (int) valorCompra;
}

// ClienteVIP.java (2 XP por R$ 1,00)
public int calcularXP(double valorCompra){
    return (int) (valorCompra * 2);
}

// ClienteCasual.java (nunca acumula)
public int calcularXP(double valorCompra){
    return 0;
}
```

`Pedido.finalizar()` chama `cliente.calcularXP(total)` sem saber qual subtipo de `Cliente` está por trás da referência — a decisão de qual versão executar é feita em tempo de execução (*dynamic binding*), o que é a essência do polimorfismo por sobrescrita.

Sobre o `ClienteCasual`: antes desta versão, um pedido sem cliente cadastrado era representado por `cliente == null`, e `Pedido.finalizar()` lançava `NullPointerException` ao chamar `cliente.calcularXP(total)`. A introdução de `ClienteCasual` (cujo `calcularXP()` sempre devolve `0`) resolve o problema aplicando o *Null Object Pattern*: em vez de tratar a ausência de cliente como um caso especial verificado com `if (cliente != null)`, o "cliente ausente" passa a ser um objeto `Cliente` como outro qualquer, apenas com um comportamento neutro. O código de `finalizar()` não precisou de nenhuma ramificação condicional — o polimorfismo por si só já absorve o caso da venda avulsa.

3.6. Polimorfismo por Sobrecarga Item 6

```
// Pedido.java
public void adicionarItem(Produto produto)
    throws EstoqueInsuficienteException{
    adicionarItem(produto, 1);          // 1 unidade por padrao
}

public void adicionarItem(Produto produto, int quantidade)
    throws EstoqueInsuficienteException{
    produto.reduzirEstoque(quantidade);
    itens.add(new ItemPedido(produto, quantidade));
}
```

As duas assinaturas de `adicionarItem` — exigidas explicitamente pelo enunciado — permitem que o chamador omita a quantidade quando deseja apenas uma unidade. Como reforço do mesmo conceito, o construtor de `Pedido` também está sobrecarregado:

```
// pedido com cliente cadastrado
public Pedido(Cliente cliente, Atendente atendente){ ... }

// venda avulsa: usa ClienteCasual por padrao
public Pedido(Atendente atendente){
    this(new ClienteCasual(), atendente);
}
```

3.7. Polimorfismo por Coerção Item 7

Este era o ponto mais frágil da primeira versão do código: o construtor de `Bebida` declarava `precoBase` como `int`, enquanto `Produto` e `Comida` usavam `double`. Isso fazia com que uma coerção (conversão implícita `int` → `double`) acontecesse "escondida" em algumas chamadas, sem que fosse uma demonstração consciente do conceito. A correção teve duas partes:

1. Padronizar `precoBase` como `double` em `Produto`, `Comida` e `Bebida`, eliminando a inconsistência de tipos.
2. Adicionar, isolado e comentado em `Main.java`, um exemplo inequívoco de coerção:

```
// Polimorfismo por Coercao: conversao implicita (widening)
// de int para double, sem necessidade de cast explicito.
int gorjetaEmReais = 5;
double totalComGorjeta =
    pedido1.calcularTotal() + gorjetaEmReais;
```

Na expressão acima, `gorjetaEmReais` (um `int`) é somado a um `double` (o retorno de `calcularTotal()`). O compilador Java promove automaticamente o valor inteiro para `double` antes da soma — sem qualquer *cast* explícito — o que caracteriza a coerção (conversão de tipo implícita), em contraste com um *casting* explícito como `(double) gorjetaEmReais`.

3.8. Interfaces Item 8

```
// Promocional.java
public interface Promocional {
    double aplicarDesconto(double valor);
}

// EventoGeekPromocao.java (10% de desconto)
public class EventoGeekPromocao implements Promocional {
    public double aplicarDesconto(double valor){
        return valor * 0.9;
    }
}
```

`Pedido` guarda uma referência do tipo `Promocional` — nunca conhece `EventoGeekPromocao` diretamente. Isso permite, no futuro, criar outras promoções (`Promocional` sendo implementada por outra classe) sem qualquer alteração em `Pedido`.

Um ponto de atenção corrigido nesta versão: o enunciado especifica que o desconto de "Dia de Evento Geek" deve incidir **apenas sobre o valor das bebidas**, não sobre o pedido inteiro. A implementação original aplicava os 10% sobre o total geral. A correção separa os subtotais antes de aplicar o desconto:

```
// Pedido.java
public double calcularTotal(){
    double totalComida = 0, totalBebida = 0;
    for (ItemPedido item : itens){
        if (item.getProduto() instanceof Bebida)
            totalBebida += item.subtotal();
        else
            totalComida += item.subtotal();
    }
    if (promocao != null){
        totalBebida = promocao.aplicarDesconto(totalBebida);
    }
    return totalComida + totalBebida;
}
```

Exemplo verificado: 1 comida de R\$ 8,00 + 1 bebida de R\$ 8,00 com a promoção ativa resulta em R\$ 8,00 + R\$ 7,20 = **R\$ 15,20** (e não R\$ 14,40, que seria o desconto aplicado indevidamente sobre o total).

4. Justificativa das Exceções Customizadas

As duas exceções exigidas pelo enunciado (RNF 3.1) foram implementadas como **checked** (estendendo `Exception`, não `RuntimeException`). A justificativa é a mesma para as duas, e segue o critério clássico (Joshua Bloch, *Effective Java*): usar exceções checked para condições que o chamador pode razoavelmente antecipar e das quais pode se recuperar; usar unchecked para erros de programação que não deveriam acontecer se o código estiver correto.

Exceção	Quando é lançada	Por que é <i>checked</i>
<code>EstoqueInsuficienteException</code>	<code>Produto.reduzirEstoque()</code> , chamada a partir de <code>Pedido.adicionarItem()</code> , quando a quantidade pedida excede o estoque disponível.	É uma condição de negócio esperada e recuperável — faltar estoque de um produto é rotina numa cafeteria. Tornar a exceção checked obriga quem chama <code>adicionarItem()</code> a decidir explicitamente o que fazer (avisar o cliente, sugerir outro item, reduzir a quantidade), em vez de deixar o erro se propagar silenciosamente até derrubar a aplicação.
<code>PontosInsuficientesException</code>	<code>ClienteVIP.pagarComXP()</code> , chamada a partir de <code>Pedido.finalizarComXP()</code> , quando o saldo de XP do cliente é menor que o necessário para cobrir o valor da compra.	Mesma lógica: um cliente VIP tentar pagar com XP insuficiente é um cenário de negócio plausível, não um bug. O caráter checked garante que o fluxo de pagamento trate essa possibilidade (por exemplo, oferecendo pagamento em

		dinheiro como alternativa) em vez de simplesmente travar o programa.
--	--	--

Em ambos os casos, o construtor recebe apenas uma `String` mensagem repassada ao construtor de `Exception`, o suficiente para descrever a falha de negócio sem expor detalhes de implementação.

5. Considerações Finais

A implementação cobre a totalidade dos oito itens do checklist de conceitos obrigatórios: classes e associações, escopo estático/dinâmico, herança simples, e as quatro formas de polimorfismo (inclusão, sobrescrita, sobrecarga e coerção), além das interfaces. Os dois pontos que a primeira revisão do código havia identificado como frágeis — a coerção não demonstrada conscientemente e o desconto aplicado sobre o pedido inteiro em vez de somente as bebidas — foram corrigidos e verificados executando o programa. Da mesma forma, o bug do cliente casual (`NullPointerException` em `finalizar()`) foi eliminado com o *Null Object Pattern*, e o método `pagarComXP()`, que antes existia mas nunca era chamado, passou a fazer parte do fluxo de `Pedido` através de `finalizarComXP()`.